

Remote Log Collection

Design a feature that lets our customer support team pull diagnostic logs from software running on a customer's own PC — with the customer's permission.

DURATION 40–45 minutes

FORMAT Discussion, whiteboard optional

CODE Not required

01

Background

We sell desktop software that customers install on PCs in their own offices and laboratories. When something goes wrong, the software writes diagnostic logs to disk on that machine.

Our support team works from a web portal. A representative can look up any customer account and see the installations registered against it — machine name, software version, when it last checked in.

The problem today. When a support representative needs those logs, they have to talk the customer through finding the right folder, zipping the files and emailing them across. It is slow, it frequently produces the wrong files, and on a bad day it turns a twenty-minute fix into a two-day exchange of emails.

02

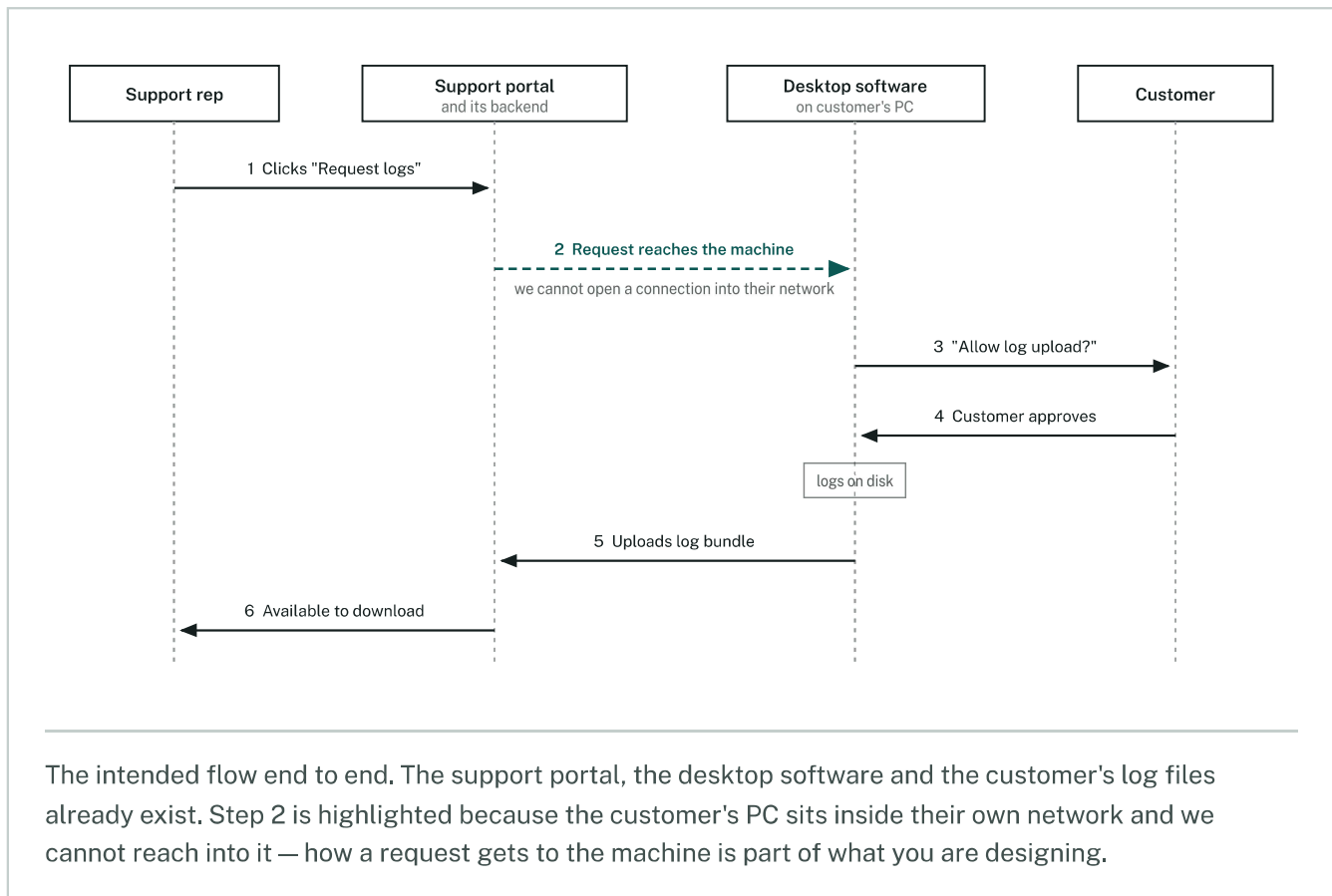
What we want to build

A button in the support portal that says **Request logs**.

When a representative clicks it, the customer is asked to approve the request on their own machine. If they approve, the desktop software gathers the relevant logs and uploads them. The representative can then download the logs from the portal and get on with diagnosing the problem.

Nothing leaves the customer's machine without their explicit consent.

How it should work



A worked example

- 1 A customer calls support: since Tuesday, their instrument results are not syncing. The representative, **Priya**, opens the customer's account in the portal and sees three installations. The caller confirms the machine is the one in the second-floor lab.
- 2 Priya opens that installation and clicks **Request logs**. She adds a short reason: "Sync failures since 12 Aug."
- 3 On the lab PC, a notification appears: "Bio-Rad Support has requested diagnostic logs from this computer. Review and approve?" It shows who asked, why, and what will be sent.
- 4 The lab manager reads it and clicks **Approve**.
- 5 The desktop software collects the relevant log files, packages them, and uploads them. The upload takes a few minutes over the site's connection.
- 6 Priya's portal page updates to show the logs have arrived. She downloads the bundle and finds the failing sync job in the first file she opens.

What already exists

Assume all of the following is already built and working. You do not need to design any of it.

Support portal	A web application used by our support team. Representatives sign in, look up customer accounts, and view the installations registered to each one.
Installation records	Every install is registered against a customer account with a machine name, software version, and the time it last contacted us.
Desktop software	Runs on the customer's PC. It already contacts our cloud service periodically for licence validation and update checks.
Log files	Written to disk on the customer's machine, rotated daily, retained locally for 30 days.
Cloud hosting	Assume ordinary cloud infrastructure is available to you — compute, managed databases, object storage, queues.

The environment you are designing for

Scale	Roughly 50,000 active installations across about 8,000 customer accounts. Around 200 support representatives.
Expected volume	Perhaps 300–500 log requests on a normal working day, concentrated in business hours across several time zones.
Network	Customer PCs sit inside the customer's own network, usually behind a firewall and often a proxy. We cannot open a connection into that network. Outbound connections from the machine to us are permitted.
Availability	Machines are switched off overnight and at weekends. Some are offline for days at a time.
Log size	A typical bundle is 5–50 MB. Occasionally one reaches 500 MB.
Sensitivity	Logs can contain patient-identifiable information. Treat them as confidential data throughout.
Consent	Non-negotiable. Nothing may be collected or transmitted without the customer approving that specific request.

What we would like you to design

COMPONENTS

What pieces make up this feature, what is each responsible for, and how do they communicate?

APIS

What endpoints are needed, who calls them, and what goes in and out?

DATA MODEL

What tables are required, with the columns that matter and how the records relate.

NON-FUNCTIONAL

Security, privacy, reliability, performance, auditability — and how your design meets them.